

Cross-Entropy Demystified

交叉熵解密

What is it? Is there any relation to the entropy concept? Why is it used for classification loss? What about the binary cross-entropy?
它是什么？与熵概念有关系吗？它为什么用于分类损失？二元交叉熵如何？



Some of us might have used the cross-entropy for calculating classification losses and wondered why we use the natural logarithm. Some might have seen the binary cross-entropy and wondered whether it is fundamentally different from the cross-entropy or not. If so, reading this article should help to demystify those questions.
有些人可能用交叉熵来计算分类损失，并想知道为什么要使用自然对数。有些人可能见过二元交叉熵，并想知道它与交叉熵是否有本质区别。如果是这样，阅读本文应该有助于解开这些疑问。

The word “cross-entropy” has “cross” and “entropy” in it, and it helps to understand the “entropy” part to understand the “cross” part.
交叉熵 "一词包含 "交叉 "和 "熵"，理解了 "熵 "的部分，才能理解 "交叉 "的部分。

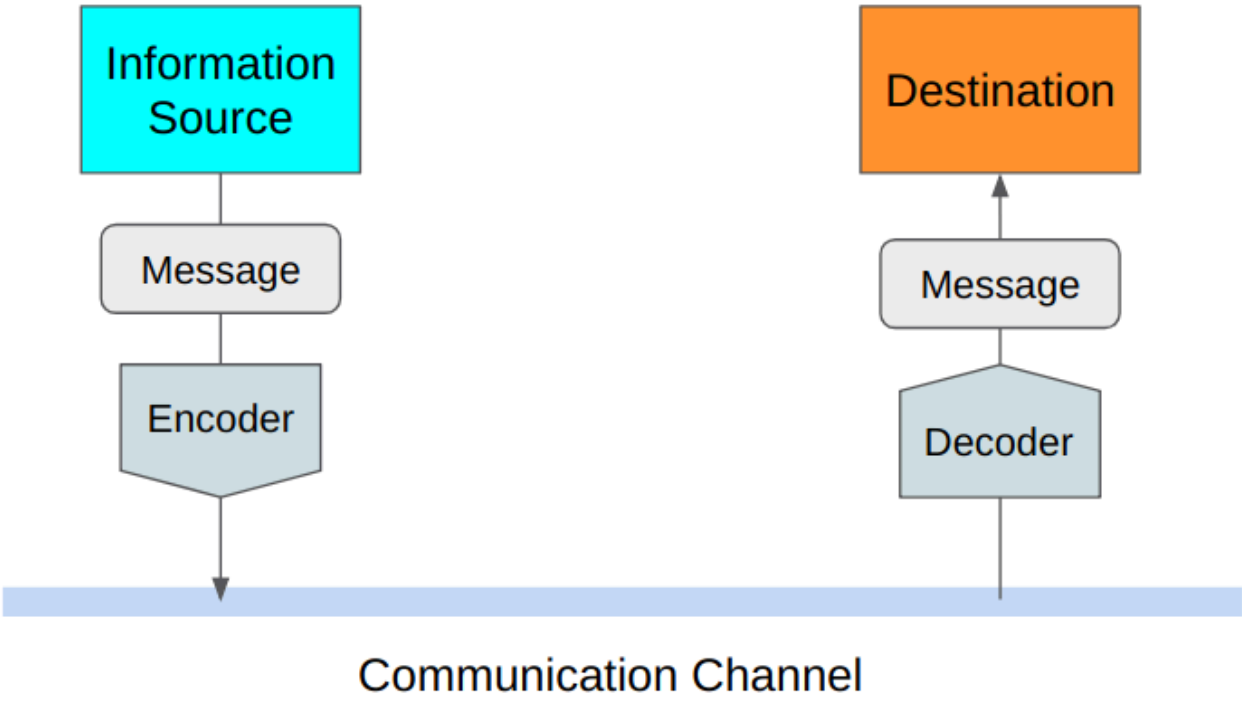
So, let’s review the entropy formula.
那么，让我们回顾一下熵公式。

Review of Entropy Formula

熵公式回顾

My article [Entropy Demystified](#) should help the understanding of the entropy concept if not already familiar with it.
如果您还不熟悉熵的概念，我的文章《熵的解密》应该有助于您理解熵的概念。

Claude Shannon (https://en.wikipedia.org/wiki/Claude_Shannon) defined the entropy to calculate the minimum encoding size as he was looking for a way to efficiently send messages without losing any information.
克劳德-香农（https://en.wikipedia.org/wiki/Claude_Shannon）定义了熵来计算最小编码大小，因为他正在寻找一种既能高效发送信息又不会丢失任何信息的方法。



As we will see below, there are various ways of expressing the entropy.
我们将在下文中看到，熵有多种表达方式。

The entropy of a probability distribution is as follows:
概率分布的熵值如下：

$$Entropy = - \sum_i P(i) \log P(i)$$

We assume that we know the probability P for each i . The term i indicates a discrete event that could mean different things depending on the scenario you are dealing.

我们假设知道每个 i 的概率 P 。术语 i 表示一个离散事件，根据你所处理的情况，它可能有不同的含义。

For continuous variables, it can be written using the integral form:

对于连续变量，可以用积分形式来写：

$$Entropy = - \int P(x) \log P(x) dx$$

Here, x is a continuous variable, and $P(x)$ is the probability density function.

这里， x 是连续变量， $P(x)$ 是概率密度函数。

In both discrete and continuous variable cases, we are calculating the expectation (average) of the negative log probability which is the theoretical minimum encoding size of the information from the event x .

无论是离散变量还是连续变量，我们计算的都是负对数概率的期望值（平均值），这是事件 x 信息的理论最小编码大小。

So, the above formula can be re-written in the expectation form as follows:

因此，上面的公式可以用期望值的形式重写如下：

$$Entropy = \mathbb{E}_{x \sim P}[-\log P(x)]$$

$x \sim P$ means that we calculate the expectation with the probability distribution P .

$x \sim P$ 表示我们用概率分布 P 计算期望值。

The entropy is also written using H as follows:

熵也可以用 H 来表示，如下所示：

$$H(P) = \mathbb{E}_{x \sim P}[-\log P(x)]$$

People use all these notations interchangeably, and we see them used in different places when we read articles and papers that use the entropy concepts. 人们交替使用所有这些符号，当我们阅读使用熵概念的文章和论文时，我们会看到它们在不同的地方被使用。

In short, the entropy tells us the theoretical minimum average encoding size for events that follow a particular probability distribution.

简而言之，熵告诉我们遵循特定概率分布的事件的理论最小平均编码大小。

As long as we know the probability distribution of anything, we can calculate the entropy for it.

只要我们知道任何事物的概率分布，就能计算出它的熵。

If we do not know the probability distribution, we can not calculate the entropy. So, we would need to estimate the probability distribution.

如果我们不知道概率分布，就无法计算熵。因此，我们需要估计概率分布。

However, what does it mean to estimate the entropy?

然而，估算熵值意味着什么呢？

Also, how accurate would that be?

另外，这样做的准确性如何？

Answering these questions naturally yields to the cross-entropy.

回答这些问题，自然就会得出交叉熵。

Estimating Entropy 估计熵

Let's assume we are reporting Tokyo's weather to New York, and we want to encode the message into the smallest possible size. We do not know the distribution of the weather before it happens. For the sake of discussion, let's assume we can find it out after observing the weather in Tokyo for some time. 假设我们要向纽约报告东京的天气情况，我们希望将信息编码成尽可能小的大小。在天气发生之前，我们并不知道它的分布情况。为了便于讨论，我们假设在观察东京的天气一段时间后就能知道。

As we do not initially know the probability distribution of Tokyo's weather, we estimate it as Q to calculate the encoding size for transmitting our weather report from Tokyo to New York.

由于我们最初并不知道东京天气的概率分布，我们将其估计为 Q ，以计算从东京向纽约传输天气报告的编码大小。

Using the estimated probability distribution Q , the estimated entropy would be:
使用估计的概率分布 Q ，估计的熵为

$$EstimatedEntropy = \mathbb{E}_{x \sim Q}[-\log Q(x)]$$

If Q is close to the real probability distribution, the above estimation tells us what the smallest average encoding size is.
如果 Q 接近真实的概率分布，上述估算就能告诉我们最小的平均编码大小是多少。

However, there are two kinds of uncertainty involved in the estimation formula.
然而，估算公式中存在两种不确定性。

As indicated by $x \sim Q$, we are using the estimated probability distribution Q to calculate the expectation which would be different from the actual probability distribution P .
如 $x \sim Q$ 所示，我们使用估计的概率分布 Q 来计算期望值，而期望值将不同于实际的概率分布 P 。
Moreover, we are estimating the minimum encoding size as $-\log Q$ which we calculate based on the estimated probability distribution Q . Hence, it will not be 100% accurate.
此外，我们根据估计的概率分布 Q 计算出的最小编码大小为 $-\log Q$ 。

As the estimated probability distribution Q affects both the expectation and encoding size estimation, the estimated entropy can be very wrong.
由于估计的概率分布 Q 会影响期望值和编码大小的估计，因此估计的熵可能会非常错误。

Alternatively, it can be close to the real entropy by coincidence because Q affects both the expectation calculation and the encoding size estimation.
另外，由于 Q 同时影响期望值计算和编码大小估计，它也可能因巧合而接近真实熵。

So, comparing the estimated entropy with the real entropy may not mean anything.
因此，将估计熵与实际熵进行比较可能毫无意义。

Like Claude Shannon, our main concern is how to make the encoding size as small as possible. So, we want to compare our encoding size with the theoretical minimum encoding size based on the entropy.
与克劳德-香农一样，我们主要关注的是如何使编码规模尽可能小。因此，我们希望将我们的编码大小与基于熵的理论最小编码大小进行比较。

If we have the real distribution P after observing the weather in Tokyo for some period, we can calculate the realized average encoding size using the probability distribution P and the actual encoding size (based on Q) used during the weather reporting.
如果我们在观察了东京一段时间的天气后得到了真实分布 P ，我们就可以利用概率分布 P 和天气预报时使用的实际编码大小（基于 Q ）计算出实现的平均编码大小。

It is called the cross-entropy between P and Q , which we can compare with the entropy.
它被称为 P 和 Q 之间的交叉熵，我们可以将其与熵进行比较。

$$CrossEntropy = \mathbb{E}_{x \sim P}[-\log Q(x)]$$
$$Entropy = \mathbb{E}_{x \sim P}[-\log P(x)]$$

We are comparing apple to apple as we use the same true distribution for both expectation calculations. We are comparing the theoretical minimum encoding and the actual encoding used in the weather reporting.
我们是在比较苹果和苹果，因为我们在两种期望值计算中使用了相同的真实分布。我们比较的是理论上的最小编码和天气预报中实际使用的编码。

In short, we are cross-checking the encoding size, which is what “cross” means in cross-entropy.
简而言之，我们正在交叉检查编码大小，这就是交叉熵中 “交叉” 的含义。

Cross-entropy ≥ Entropy

交叉熵 ≥ 熵

Commonly, the cross-entropy is expressed using H as follows:
通常，交叉熵用 H 表示如下：

$$H(P, Q) = \mathbb{E}_{x \sim P}[-\log Q(x)]$$

$H(P, Q)$ means that we calculate the expectation using P and the encoding size using Q . As such, $H(P, Q)$ and $H(Q, P)$ is not necessarily the same except when $Q=P$, in which case $H(P, Q) = H(P, P) = H(P)$ and it becomes the entropy itself.
因此， $H(P, Q)$ 和 $H(Q, P)$ 不一定相同，除非当 $Q=P$ 时，在这种情况下， $H(P, Q) = H(P, P) = H(P)$ ，它本身就是熵。

This point is subtle but essential. For the expectation, we should use the true probability P as that tells the distribution of events. For the encoding size, we should use Q as that is used to encode messages.
这一点很微妙，但却至关重要。对于期望值，我们应该使用真实概率 P ，因为它说明了事件的分布情况。对于编码大小，我们应该使用 Q ，因为 Q 是用来对信息进行编码的。

Since the entropy is the theoretical minimum average size, the cross-entropy is higher than or equal to the entropy but not less than that. 由于熵是理论上的最小平均尺寸，因此交叉熵高于或等于熵，但不小于熵。

In other words, if our estimate is perfect, $Q = P$ and, hence, $H(P, Q)=H(P)$. Otherwise, $H(P, Q) > H(P)$. 换句话说，如果我们的估计是完美的，那么 $Q = P$ ，因此 $H(P, Q) = H(P)$ 。否则， $H(P, Q) > H(P)$ 。

By now, the connection between the entropy and the cross-entropy should be clear. Let’s talk about why we use the cross-entropy for classification loss function. 现在，熵和交叉熵之间的联系应该很清楚了。下面我们来谈谈为什么要使用交叉熵作为分类损失函数。

Cross-Entropy as a Loss Function

作为损失函数的交叉熵

Let’s say we have a dataset of animal images and there are five different animals. Each image has only one animal in it. 假设我们有一个动物图片数据集，其中有五种不同的动物。每张图片中只有一种动物。



Source: <https://www.freeimages.com/>
来源： <https://www.freeimages.com/>

Each image is labeled with the corresponding animal using the one-hot encoding. 每张图像都使用单次编码标记相应的动物。

Animal	Label
Dog	[1 0 0 0 0]
Fox	[0 1 0 0 0]
Horse	[0 0 1 0 0]
Eagle	[0 0 0 1 0]
Squirrel	[0 0 0 0 1]

We can treat one hot encoding as a probability distribution for each image. Let’s see a few examples. 我们可以将一种热编码视为每幅图像的概率分布。让我们来看几个例子。

The probability distribution of the first image being a dog is 1.0 (=100%). 第一张图片是狗的概率分布为 1.0 (=100%)。

$$P_1(dog)$$
$$P_1(fox)$$
$$P_1(horse)$$
$$P_1(eagle)$$
$$P_1(squirrel)$$

$$= 1$$
$$= 0$$
$$= 0$$
$$= 0$$
$$= 0$$

For the second image, the label tells us that it is a fox with 100% certainty. 至于第二张图片，标签告诉我们这是一只狐狸，这一点百分之百确定。

$$\begin{aligned}
 P_2(dog) &= 0 \\
 P_2(fox) &= 1 \\
 P_2(horse) &= 0 \\
 P_2(eagle) &= 0 \\
 P_2(squirrel) &= 0
 \end{aligned}$$

...and so on.
.....等等。

As such, the entropy of each image is all zero.
因此，每幅图像的熵均为零。

$$\begin{aligned}
 H(P_1) &= 0 \\
 H(P_2) &= 0 \\
 H(P_3) &= 0 \\
 H(P_4) &= 0 \\
 H(P_5) &= 0
 \end{aligned}$$

In other words, one-hot encoded labels tell us what animal each image has with 100% certainty. It is not like the first image can be a dog for 90% and a cat for 10%. It is always a dog, and there will be no surprise.
换句话说，单击编码标签可以 100%确定地告诉我们每张图片上的动物是什么。并不是说第一张图片 90% 是狗，10% 是猫。它总是狗，不会有任何意外。

Now, let's say we have a machine learning model that classifies those images. When we have not adequately trained the model, it may classify the first image (dog) as follows:

现在，假设我们有一个机器学习模型来对这些图像进行分类。当我们没有对模型进行充分训练时，它可能会对第一幅图像（狗）进行如下分类：

$$Q_1 = [0.4 \quad 0.3 \quad 0.05 \quad 0.05 \quad 0.2]$$

The model says the first image is 40% for a dog, 30% for a fox, 5% for a horse, 5% for an eagle and 20% for a squirrel. This estimation is not very precise or confident about what animal the first image has.

模型显示，第一幅图像中狗占 40%，狐狸占 30%，马占 5%，鹰占 5%，松鼠占 20%。这种估计对于第一幅图像中的动物并不精确，也不确定。

In contrast, the label gives us the exact distribution of the first image's animal class. It tells us it is a dog with 100% certainty.
相比之下，标签则为我们提供了第一张图片动物类别的准确分布。它能百分之百地告诉我们这是一只狗。

$$P_1 = [1 \quad 0 \quad 0 \quad 0 \quad 0]$$

So, how well was the model's prediction? We can calculate the cross-entropy as follows:

那么，模型的预测结果如何呢？我们可以如下计算交叉熵：

$$\begin{aligned}
 H(P_1, Q_1) &= - \sum_i P_1(i) \log Q_1(i) \\
 &= -(1 \log 0.4 + 0 \log 0.3 + 0 \log 0.05 + 0 \log 0.05 + 0 \log 0.2) \\
 &= -\log 0.4 \\
 &\approx 0.916
 \end{aligned}$$

This is higher than the zero entropy of the label but we do not have an intuitive sense of what this value means. So, let's see another cross-entropy value for comparison.

这比标签的零熵值要高，但我们并不能直观地理解这个值的含义。因此，我们再来看看另一个交叉熵值，以作比较。

After the model is well trained, it may produce the following prediction for the first image.

模型训练有素后，可以对第一幅图像做出如下预测。

$$Q_1 = [0.98 \quad 0.01 \quad 0 \quad 0 \quad 0.01]$$

As below, the cross-entropy is much lower than before.
如下图所示，交叉熵比以前低得多。

$$\begin{aligned} H(P_1, Q_1) &= - \sum_i P_1(i) \log Q_1(i) \\ &= -(1 \log 0.98 + 0 \log 0.01 + 0 \log 0 + 0 \log 0 + 0 \log 0.01) \\ &= -\log 0.98 \\ &\approx 0.02 \end{aligned}$$

The cross-entropy compares the model's prediction with the label which is the true probability distribution. The cross-entropy goes down as the prediction gets more and more accurate. It becomes zero if the prediction is perfect. As such, the cross-entropy can be a loss function to train a classification model. 交叉熵将模型的预测与标签（即真实的概率分布）进行比较。随着预测越来越准确，交叉熵会下降。如果预测是完美的，交叉熵就会变为零。因此，交叉熵可以作为训练分类模型的损失函数。

Notes on Nats vs. Bits

国家队对阵比特队的比赛笔记

In machine learning, we use **base e** instead of **base 2** for multiple reasons (one of them being the ease of calculating the derivative). 在机器学习中，我们使用基 e 而不是基 2 有多种原因（其中之一是便于计算导数）。

The change of the logarithm base does not cause any problem since it changes the magnitude only. 改变对数基数不会造成任何问题，因为它只改变了大小。

$$\log_2 x = \frac{\log_e x}{\log_e 2}$$

For example, using the cross-entropy as a classification cost function still makes sense as we only care about reducing it by training the model better and better. 例如，使用交叉熵作为分类成本函数仍然是合理的，因为我们只关心通过训练模型来降低交叉熵。

As a side note, the information unit with the **base e** logarithm is **nats** whereas the information unit with the **base 2** logarithm is called **bits**. The amount of information in **1 nat** comes from an event that happens with 1/e probability. 顺便提一下，以 e 为底对数的信息单位是纳特，而以 2 为底对数的信息单位称为比特。1 纳特的信息量来自以 1/e 的概率发生的事件。

$$\begin{aligned} 1 &= -\log P \\ \therefore P &= \frac{1}{e} \end{aligned}$$

The **base e** logarithms are less intuitive than the **base 2** logarithms. The amount of information in 1 bit comes from an event that happens with 1/2 probability. If we can encode a piece of information in 1 bit, one such message can reduce 50% uncertainty. The same analogy is not easy to make with the **base e**, which is why the **base 2** logarithm is often used to explain the information entropy concept. However, the machine learning application uses the **base e** logarithm for implementation convenience. 以 e 为底的对数不如以 2 为底的对数直观。1 比特的信息量来自以 1/2 概率发生的事件。如果我们能用 1 比特编码一条信息，那么一条这样的信息就能减少 50% 的不确定性。同样的类比在以 e 为底的情况下不易实现，这也是为什么人们经常使用以 2 为底的对数来解释信息熵概念的原因。不过，机器学习应用使用基 e 对数是为了执行方便。

Binary Cross-Entropy

二元交叉熵

We can use the binary cross-entropy for binary classification where we have yes/no answer. For example, there are only dogs or cats in images. 我们可以将二元交叉熵用于有是/否答案的二元分类。例如，图像中只有狗或猫。

For the binary classifications, the cross-entropy formula contains only two probabilities: 对于二元分类，交叉熵公式只包含两个概率：

$$\begin{aligned} H(P, Q) &= - \sum_{i=(cat, dog)} P(i) \log Q(i) \\ &= -P(cat) \log Q(cat) - P(dog) \log Q(dog) \end{aligned}$$

Using the following relationship
利用以下关系

$$P(dog) = (1 - P(cat))$$

, we can write the cross-entropy as follows:
, 我们可以将交叉熵写法如下:

$$H(P, Q) = -P(cat) \log Q(cat) - (1 - P(cat)) \log(1 - Q(cat))$$

We can further simplify it by changing the symbols slightly:
我们可以通过稍微改变符号来进一步简化它:

$$P = P(cat)$$
$$\hat{P} = Q(cat)$$

Then, the binary cross-entropy formula becomes:
那么, 二元交叉熵公式就变成了

$$BinaryCrossEntropy = -P \log \hat{P} - (1 - P) \log(1 - \hat{P})$$

, which may look more familiar to some of us.
这对我们中的一些人来说可能更熟悉。

In short, the binary cross-entropy is a cross-entropy with two classes. Other than that, they are the same concept.
简而言之, 二元交叉熵就是包含两个类别的交叉熵。除此之外, 它们是同一个概念。

Now that we know the concepts well, the entropy of the cross-entropy concept should be zero to all of us.
既然我们已经对这些概念了如指掌, 那么对于我们所有人来说, 交叉熵概念的熵值应该为零。